Intermediate — an HTML + JavaScript frontend — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: "frontend" sometimes means a genuinely separate HTML page whose JavaScript calls your API over `fetch()` — the real backend-plus-frontend split a production app, or a React client, actually uses in practice. **If not:** you'd only ever know Streamlit, and you couldn't connect a real web frontend to your pipeline, or even explain clearly how one would work.

The backend: a page and a JSON API

```
# practice/app_frontend.py   —   uvicorn app_frontend:app --reload
from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from pydantic import BaseModel

from pipeline import classify

app = FastAPI()

PAGE = """
<!doctype html>
<html>
<body>
  <h2>Sentiment</h2>
  <input id="text" placeholder="Type something">
  <button onclick="go()">Classify</button>
  <p id="out"></p>
  <script>
    async function go() {
      const text = document.getElementById("text").value;
      const res = await fetch("/api/classify", {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify({text: text})
      });
      const data = await res.json();
      document.getElementById("out").innerText = "Result: " + data.label;
```

```
    }
  </script>
</body>
</html>
"""

class ClassifyRequest(BaseModel):
    text: str

@app.get("/", response_class=HTMLResponse)
def page():
    return PAGE

@app.post("/api/classify")
def api(body: ClassifyRequest):
    return {"label": classify(body.text)}
```

(`pipeline.py` has the same `classify(text)` from the Basic exercise.)

How it works

- **`GET /`** serves a plain HTML page — a text box, a button, and an empty `<p>`.
- **The JavaScript `go()`** reads the box, `fetch`es `POST /api/classify` with a JSON body, awaits the JSON reply, and writes `data.label` into the page. No page reload — the result appears in place.
- **`POST /api/classify`** is your typed JSON endpoint — the exact one from the FastAPI exercise. The page and a React app would call it the same way.
- **The key stays on the server.** The browser only ever talks to *your* `/api/classify`; that endpoint talks to the model. The API key is never in the HTML or the JavaScript — that's the whole reason you don't call the model from the browser directly.

Testing the endpoint (the page you open in a browser)

```
# practice/test_frontend.py   —   python -m practice.test_frontend
from fastapi.testclient import TestClient

from app_frontend import app

def test_page_and_api() -> None:
    client = TestClient(app)
    page = client.get("/")
    api = client.post("/api/classify", json={"text": "I love it"}).json()
    result = "OK"
```

```
    if page.status_code != 200 or "fetch" not in page.text:
        result = "WRONG"
    if api != {"label": "positive"}:
        result = "WRONG"
    print(f"case 1 (page served + api works) -> {result}")

if __name__ == "__main__":
    test_page_and_api()
```

Expected: `GET /` returns the HTML page (containing the `fetch` script); `POST /api/classify` `{"text": "I love it"}` returns `{"label": "positive"}`. Run `uvicorn app_frontend:app --reload`, open `http://127.0.0.1:8000`, type, click — the answer appears without a reload.

Why this matters: this is how any real web frontend — plain JS, React, Vue — talks to an AI backend: the page `fetch`es your JSON API, the key stays server-side. For dynamic pages with almost no JavaScript, **HTMX** does the same with an `hx-post` attribute on the form and HTML fragments from the server.